

Deep Reinforcement Learning Formulation for Orbital Edge Computing Depth Selection

1 Problem Overview

We consider a deep reinforcement learning (DRL) approach for adaptive compression-depth selection in an orbital edge computing (OEC) system. Earth-observation satellites generate image-processing tasks while passing over predefined areas of interest (AOIs). Each task consists of a collection of images that must be encoded onboard the source satellite and transmitted through a time-varying low-Earth-orbit (LEO) satellite network to a ground base station.

Each task $k \in \mathcal{K}$ is characterized by a source satellite s_k , destination ground station g_k , arrival time a_k , number of images N_k , importance weight w_k , and soft deadline d_k . Images are compressed using a residual-quantized variational autoencoder (RQ-VAE). The compression depth q_k is selected from

$$\mathcal{D} = \{2, 4, 8, 16\}.$$

Increasing the compression depth improves the measured downstream image quality in the current system but also increases the number of bits that must be transmitted. The depth-selection problem therefore creates a tradeoff between image quality and network resource consumption.

The DRL agent learns a task-level policy that balances image quality, timely delivery, coverage, and resource cost under time-varying network connectivity and capacity constraints. The implementation uses Proximal Policy Optimization (PPO) with action masking. The following sections describe the satellite-network environment, task-generation process, DRL formulation, unified utility, reward function, PPO training procedure, evaluation protocol, and relevant implementation details.

2 Satellite Network Environment

2.1 LEO Constellation and Simulation Horizon

The default DRL environment uses the Kuiper-630 LEO constellation. The constellation contains 34 orbital planes with 34 satellites per plane, for a total of

$$|\mathcal{S}| = 34 \times 34 = 1156$$

satellites. The satellites orbit at an altitude of 630 km with an inclination of 51.9° .

The network is simulated for five hours using scheduling slots of duration

$$\Delta t = 30 \text{ s.}$$

The total configured simulation duration is

$$T_{\text{sim}} = 18000 \text{ s.}$$

The implementation defines

$$N_{\text{slots}} = \frac{18000}{30} + 1 = 601$$

discrete slot indices, including the initial slot at time zero.

2.2 Time-Varying Network Topology

The satellite network is represented by a time-varying graph

$$\mathcal{G}_t = (\mathcal{V}, \mathcal{E}_t),$$

where the set of feasible communication links $\mathcal{E}_t$ changes as the satellites move.

Each satellite is assigned inter-satellite links (ISLs) according to a +Grid structure. Satellites connect to their neighboring satellite in the same orbital plane and to the same-index satellite in the adjacent orbital plane. Candidate ISLs are enabled or disabled in each slot according to line-of-sight and maximum-range feasibility constraints.

The configured maximum ISL range is 5016 km. An ISL is also considered blocked if its line of sight passes below the configured Earth-grazing clearance.

Ground-to-satellite links (GSLs) are available when a satellite is visible from a ground station at an elevation angle of at least

$$20^\circ.$$

The simulation contains four ground stations:

- Tokyo,
- New York,
- Sao Paulo,
- Sydney.

For each slot and destination ground station, shortest-path routing is computed over the currently feasible network using Dijkstra's algorithm. Physical link distance is used as the graph edge weight. The resulting path therefore minimizes total propagation distance and, equivalently under a constant propagation speed, propagation delay.

A task can transmit during a slot only if a feasible route exists between its source satellite and destination ground station.

2.3 Network Capacities

The default DRL experiments use the ground-station-limited scenario. Its configured rates are

$$R_{\text{ISL}} = 100 \text{ Mbps},$$

$$R_{\text{GSL}} = 100 \text{ Mbps},$$

and

$$R_{\text{GS}} = 2 \text{ Mbps}.$$

Here, R_{GS} represents the aggregate receiving capacity of a ground station across satellites communicating with that station.

For an edge or capacity resource e with rate R_e , the capacity available during one scheduling slot is

$$C_e = R_e \Delta t.$$

The DRL training procedure can override the ground-station rate to expose the policy to multiple capacity regimes while retaining the same underlying satellite topology and task model.

2.4 Areas of Interest

The environment contains eight fixed AOIs:

- California fires,
- Amazon basin,
- Sahel,
- Ganges delta,
- Barrier reef,
- Dnipro basin,
- Tohoku coast,
- Central Andes.

An AOI becomes observable when a satellite reaches an elevation angle of at least

$$40^\circ$$

relative to that AOI.

2.5 Task Generation

A task is generated when at least one satellite observes an AOI above the configured imaging-elevation threshold and the AOI's cooldown period has elapsed.

The cooldown is

$$1800 \text{ s.}$$

If multiple satellites can observe the AOI during the same slot, the satellite with the highest elevation angle is selected as the source satellite.

The destination ground station is chosen as the ground station geographically nearest to the AOI.

Each task is represented as

$$k = (s_k, g_k, a_k, N_k, w_k, d_k),$$

where:

- s_k is the source satellite,
- g_k is the destination ground station,
- a_k is the arrival slot,

- N_k is the number of images,
- w_k is the task importance,
- d_k is the absolute soft deadline.

Under the default ground-station-limited scenario,

$$N_k \sim \text{UniformInteger}(100000, 300000).$$

Task importance is sampled uniformly from

$$w_k \in \{1, 2, 3\}.$$

The deadline offset is sampled between 1800 and 3600 seconds after task arrival:

$$d_k = a_k \Delta t + D_k,$$

where

$$D_k \sim \text{UniformInteger}(1800, 3600).$$

At most 64 tasks are generated in an episode.

2.6 Onboard Encoding

Images become available for transmission gradually as they are encoded onboard the source satellite.

The measured encoding time per image is

$$t_{\text{enc}} = 12.34 \text{ ms.}$$

Therefore, the encoding rate is approximately

$$R_{\text{enc}} = \frac{1}{0.01234} \approx 81 \text{ images/s.}$$

If task k arrived at slot a_k , the implementation determines the number of encoded images available at absolute time t by

$$N_k^{\text{encoded}}(t) = \text{clip}[(t - a_k \Delta t) R_{\text{enc}}, 0, N_k].$$

2.7 RQ-VAE Compression Depth

The agent selects a compression depth from

$$\mathcal{D} = \{2, 4, 8, 16\}.$$

The payload size per image at depth q is

$$b_q = 88q \text{ bytes.}$$

The unified utility uses measured downstream segmentation quality obtained from the FLAIR validation set. The quality values are floor-anchored using the measured downstream segmentation results.

Depth q	Payload per Image	Quality s_q
2	176 B	0.182463
4	352 B	0.262156
8	704 B	0.312165
16	1408 B	0.360328

Table 1: RQ-VAE depth-dependent payload and downstream quality used by the DRL environment.

3 Deep Reinforcement Learning Formulation

The depth-selection problem is formulated as a sequential decision problem. The underlying satellite network evolves in fixed 30-second slots, while the DRL agent operates at task decision points. An agent decision is requested when an active task has not yet committed to a compression depth.

Once a task commits to a depth, subsequent transmission for that task is executed automatically by the shared satellite-network simulator.

3.1 Agent

The DRL agent controls task-level compression-depth commitment as well as defer and drop decisions.

When an uncommitted task reaches the front of the decision queue, the agent receives a compact observation describing the task and its current context. It then selects one of six discrete actions.

Once a compression depth has been selected for a task, that depth remains fixed for the remainder of the task.

3.2 Observation Space

The agent receives a 16-dimensional continuous observation

$$\mathbf{o}_t \in \mathbb{R}^{16}.$$

The observation consists of 12 task-specific features and four environment-context features. For current task k , the first feature is the fraction of images remaining:

$$o_1 = \frac{N_k - r_k}{N_k},$$

where r_k is the number of images already delivered.

The second feature is the log-scaled task size:

$$o_2 = \frac{\log_{10}(\max(N_k, 1))}{6}.$$

The third feature is normalized task importance:

$$o_3 = \frac{w_k}{3}.$$

The fourth feature represents normalized time to deadline:

$$o_4 = \text{clip}\left(\frac{d_k - t}{D_{\max}}, -2, 2\right),$$

where

$$D_{\max} = 3600 \text{ s.}$$

The fifth feature is current task freshness:

$$o_5 = \phi_k(t).$$

The sixth feature is the fraction of task images that have been encoded:

$$o_6 = \text{clip} \left(\frac{N_k^{\text{encoded}}(t)}{N_k}, 0, 1 \right).$$

The seventh feature indicates whether a feasible route currently exists:

$$o_7 = \begin{cases} 1, & \text{if a route exists,} \\ 0, & \text{otherwise.} \end{cases}$$

The eighth feature is normalized route length:

$$o_8 = \begin{cases} |P_{k,t}|/30, & \text{if a route exists,} \\ 1, & \text{otherwise,} \end{cases}$$

where $|P_{k,t}|$ denotes the number of capacity-resource entries associated with the current route.

The next four features form a one-hot encoding of the committed compression depth:

$$(o_9, o_{10}, o_{11}, o_{12}).$$

They correspond to depths

$$(2, 4, 8, 16).$$

For an uncommitted task, all four depth indicators are zero.

The final four features provide environment-level context.

Normalized simulation time is

$$o_{13} = \frac{t_{\text{slot}}}{N_{\text{slots}}}.$$

The normalized number of active tasks is

$$o_{14} = \frac{|\mathcal{K}_t^{\text{active}}|}{16}.$$

The normalized number of active tasks sharing the current task's destination ground station is

$$o_{15} = \frac{|\{j \in \mathcal{K}_t^{\text{active}} : g_j = g_k\}|}{8}.$$

The final feature is the mean freshness of all currently active tasks:

$$o_{16} = \frac{1}{|\mathcal{K}_t^{\text{active}}|} \sum_{j \in \mathcal{K}_t^{\text{active}}} \phi_j(t),$$

with a value of one when no active tasks are present.

The implementation represents this observation using a `Gymnasium Box` space with shape (16,) and component bounds $[-10, 10]$.

The observation does not explicitly contain the configured ground-station bandwidth, individual residual link capacities, or a direct congestion measurement. Network state is represented indirectly through route availability, route length, active-task count, destination contention, and task freshness.

3.3 Action Space

The action space is

$$\mathcal{A} = \{0, 1, 2, 3, 4, 5\}.$$

The actions correspond to

$$\begin{aligned} 0 &\rightarrow q = 2, \\ 1 &\rightarrow q = 4, \\ 2 &\rightarrow q = 8, \\ 3 &\rightarrow q = 16, \\ 4 &\rightarrow \text{defer}, \\ 5 &\rightarrow \text{drop}. \end{aligned}$$

Actions 0 through 3 commit the current task permanently to the corresponding compression depth.

The defer action leaves the task uncommitted and removes it from the remaining decision queue for the current network slot. The task can therefore return as a decision point in a later slot.

The drop action permanently abandons the task. A dropped task is excluded from subsequent transmission and scheduling decisions.

3.4 Action Masking

Action masking prevents the policy from committing a task to a compression depth when no feasible route currently exists.

Let

$$m_t(a) \in \{0, 1\}$$

indicate whether action a is valid.

When a feasible route exists,

$$m_t(a) = 1 \quad \forall a \in \mathcal{A}.$$

When no route exists,

$$m_t(a) = \begin{cases} 1, & a \in \{4, 5\}, \\ 0, & a \in \{0, 1, 2, 3\}. \end{cases}$$

Thus, defer and drop are always available, while depth commitment requires current route feasibility.

3.5 Transition Process

At each network slot, the environment constructs the set of active tasks:

$$\mathcal{K}_t^{\text{active}} = \{k : a_k \leq t, r_k < N_k, k \text{ not dropped}\}.$$

Active tasks are ordered by deadline.

If the next task has no committed compression depth, execution pauses and the DRL agent is asked to select an action.

Once a depth is committed, transmission is performed using the shared network execution logic also used by the non-RL schedulers.

For task k , the simulator first obtains the current shortest path

$$P_{k,t}$$

from source satellite s_k to destination ground station g_k .

The path is converted into capacity resources. These include each traversed ISL, the final satellite-to-ground link, and the aggregate receiving capacity of the destination ground station.

For resource e , slot capacity is

$$C_e = R_e \Delta t.$$

At depth q_k , one image contains

$$B_{q_k} = 8b_{q_k}$$

bits.

The number of images that can be delivered during a slot is limited by both the number of encoded but undelivered images and the bottleneck residual capacity along the path:

$$y_{k,t} = \min \left\{ N_k^{\text{encoded}}(t) - r_k, \min_{e \in P_{k,t}} \frac{C_{e,t}^{\text{res}}}{B_{q_k}} \right\}.$$

The corresponding number of bits is deducted from the residual capacity of every resource along the path.

After transmission,

$$r_k \leftarrow r_k + y_{k,t}.$$

The delivery arrival time is taken to be the end of the current scheduling slot plus propagation delay:

$$t_{\text{arrive}} = (t_{\text{slot}} + 1)\Delta t + d_{k,t}^{\text{prop}}.$$

The delivery's utility is evaluated at this arrival time.

Tasks that have already committed to a depth may execute automatically while the simulator advances. When the current slot's decision queue is exhausted, the environment advances to the next slot and continues until another uncommitted task requires an agent decision or the episode ends.

3.6 Episode Definition

One episode spans the complete configured satellite-network simulation.

The environment terminates when the slot index reaches the configured simulation horizon. Under the default configuration, the underlying network scenario covers five hours.

The environment does not use a separate truncation condition.

4 Unified Utility and Reward Formulation

4.1 Task Utility

The unified utility defines the realized value of task k as

$$U_k = \bar{w}_k \left[\omega_Q s_{q_k} \hat{G}_k - \omega_T T_k - \omega_E C_k \right].$$

Here,

- $\bar{w}_k$ is normalized task importance,
- s_{q_k} is downstream quality at compression depth q_k ,

- $\widehat{G}_k$ is freshness-weighted concave coverage,
- T_k is tardiness,
- C_k is normalized resource cost.

In unified mode,

$$\bar{w}_k = \frac{w_k}{\max(\{1, 2, 3\})} = \frac{w_k}{3}.$$

The utility weights are

$$\omega_Q = 1.0, \quad \omega_T = 0.25, \quad \omega_E = 0.05, \quad \omega_F = 0.10.$$

4.2 Downstream Quality

The unified objective uses downstream semantic-segmentation quality.

Let m_q denote the measured mIoU at compression depth q , m_{ref} the uncompressed/reference segmentation mIoU, and m_{floor} the segmentation performance when the relevant image information is unavailable.

The floor-anchored quality score is

$$s_q = \frac{m_q - m_{\text{floor}}}{m_{\text{ref}} - m_{\text{floor}}}.$$

The measured values used by the current four depth actions are

$$s_2 = 0.182463, \quad s_4 = 0.262156, \quad s_8 = 0.312165, \quad s_{16} = 0.360328.$$

4.3 Freshness

Task freshness remains one through the soft deadline and then decays exponentially:

$$\phi_k(t) = \begin{cases} 1, & t \leq d_k, \\ \exp[-\alpha(t - d_k)], & t > d_k. \end{cases}$$

The configured decay rate is

$$\alpha = \frac{1}{300} \text{ s}^{-1}.$$

Thus, an image delivered on time receives full freshness credit, while the value of increasingly late deliveries decays exponentially.

4.4 Concave Coverage

Let

$$\rho_k = \frac{r_k}{N_k}$$

denote the delivered fraction of task k .

Unified utility uses a concave coverage function so that the first portion of a task delivered has greater marginal value than the final portion.

The implementation uses four piecewise-linear segments:

$$(\Delta\rho_j, m_j) \in \{(0.25, 2.00), (0.25, 1.12), (0.25, 0.60), (0.25, 0.28)\}.$$

Each segment covers one quarter of the task, while its slope decreases as coverage increases.

For a delivery covering task fraction $\Delta\rho$ at freshness $\phi_k(t)$, the incremental coverage gain within segment j is

$$\Delta\hat{G}_k = m_j\phi_k(t)\Delta\rho.$$

The implementation fills these segments chronologically. Consequently, early coverage of a previously unserved task receives greater marginal utility than the final portion of an already well-covered task.

4.5 Tardiness

For an image arriving at time t , the normalized tardiness coefficient is

$$\tau_k(t) = \min\left(1, \frac{\max(0, t - d_k)}{T_{\text{ref}}}\right),$$

where

$$T_{\text{ref}} = 300 \text{ s.}$$

A delivery at or before the deadline therefore has zero tardiness penalty. After the deadline, the coefficient grows linearly until reaching one.

4.6 Resource Cost

The resource-cost coefficient for compression depth q is

$$c_q = \omega_{\text{tx}} \frac{b_q}{b_{\text{max}}} + \omega_{\text{enc}},$$

where

$$\omega_{\text{tx}} = 0.5, \quad \omega_{\text{enc}} = 0.5.$$

The first component increases with transmitted payload size. The second represents the fixed per-image encoding component.

Because the measured encoding time is effectively constant across the available compression depths, the encoding component does not distinguish between depth choices, whereas the transmission component increases with depth.

4.7 Incremental Realized Utility

Suppose a delivery contains a fraction

$$\Delta\rho = \frac{y_{k,t}}{N_k}$$

of task k .

Its incremental realized task utility is

$$\Delta U_k = \bar{w}_k \left[\omega_Q s_{q_k} \Delta\hat{G}_k - \omega_T \tau_k(t) \Delta\rho - \omega_E c_{q_k} \Delta\rho \right].$$

The task's delivered utility accumulates these contributions over all of its deliveries.

4.8 Fairness

After task utilities have been accumulated, the final unified system utility is

$$U = \sum_{k \in \mathcal{K}} U_k + \omega_F |\mathcal{K}| \min_{k \in \mathcal{K}} \hat{U}_k.$$

The normalized task utility used for fairness is

$$\hat{U}_k = \frac{U_k}{\bar{w}_k \omega_Q s_{\max}},$$

where

$$s_{\max} = \max_{q \in \mathcal{D}} s_q.$$

This normalization makes the fairness floor less dependent on the absolute scale and importance weight of a task.

The minimum normalized task utility

$$\min_k \hat{U}_k$$

acts as a maximin fairness surrogate. It discourages a scheduler from obtaining high aggregate utility while completely starving an individual task.

4.9 Jain Fairness Diagnostic

The implementation also reports Jain's fairness index:

$$J = \frac{\left(\sum_k \hat{U}_k\right)^2}{|\mathcal{K}| \sum_k \hat{U}_k^2}.$$

Jain's index is reported as a diagnostic metric only. It is not directly optimized by the unified objective.

4.10 Backlog Potential

In addition to realized task utility, PPO receives a potential-based reward shaping term based on the remaining network backlog.

Let B_t denote the estimated remaining number of bits across active, non-dropped tasks.

For a task with committed depth q_k , its remaining backlog is estimated using that depth's payload size. For an uncommitted task, the implementation uses depth 16 when estimating backlog.

The potential is

$$\Phi_t = -10^{-3} \frac{B_t}{10^{11}}.$$

A smaller backlog makes this negative potential closer to zero.

4.11 PPO Reward

Let

$$\Delta J_t = U_k^{\text{after}} - U_k^{\text{before}}$$

represent the immediate change in the current task’s delivered utility associated with the current decision and execution.

The PPO reward is

$$r_t = \Delta J_t + \gamma_{\Phi} \Phi_{t+1} - \Phi_t,$$

where

$$\gamma_{\Phi} = 0.999.$$

The backlog potential provides an additional learning signal that encourages reduction of outstanding network work.

An important distinction is that the final schedule-level maximin fairness bonus is computed when the complete collection of tasks is evaluated. It is not directly added to the per-step PPO reward.

Thus, PPO directly receives realized quality, freshness/coverage, tardiness, resource-cost, and backlog-shaping signals, while the global fairness term is applied to the completed schedule during unified-utility evaluation.

5 PPO Implementation and Training

5.1 MaskablePPO

The policy is trained using MaskablePPO from the `sb3-contrib` extension to Stable-Baselines3.

MaskablePPO combines PPO with state-dependent action masks. This ensures that actions identified as invalid by the environment are excluded from the policy’s available choices.

PPO optimizes a clipped policy objective. Let

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|\mathbf{o}_t)}{\pi_{\theta_{\text{old}}}(a_t|\mathbf{o}_t)}$$

denote the probability ratio between the updated and previous policies.

Given estimated advantage $\hat{A}_t$, the standard clipped PPO surrogate objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

The action mask further restricts the policy distribution to actions that are valid at the current decision point.

5.2 Policy Network

The implementation uses an MLP policy.

The configured hidden-layer architecture is

$$[128, 128].$$

Conceptually, the observation is processed as

$$16 \rightarrow 128 \rightarrow 128 \rightarrow \text{policy/value outputs}.$$

Parameter	Value
Algorithm	MaskablePPO
Policy	MLPPolicy
Observation dimension	16
Number of actions	6
Hidden architecture	[128, 128]
Rollout length per environment	2048
Batch size	256
Learning rate	3×10^{-4}
Discount factor γ	0.999
GAE parameter λ	0.95
Entropy coefficient	0.01
Nominal training timesteps	5,000,000

Table 2: MaskablePPO training configuration.

5.3 Training Hyperparameters

The principal PPO hyperparameters are shown in Table 2.

5.4 Training Distribution

Training exposes the policy to three ground-station capacity settings:

$$R_{\text{GS}} \in \{1.5, 2.0, 2.5\} \text{ Mbps.}$$

For each capacity setting, task-generation seeds

$$0, 1, \dots, 99$$

are used.

This produces

$$3 \times 100 = 300$$

training environments.

The seed controls the deterministic pseudo-random generation of task properties such as image counts, task importance, and deadline offsets.

The environments are combined using a vectorized environment.

Each environment contributes

$$n_{\text{steps}} = 2048$$

transitions to a rollout.

Therefore, a complete vectorized rollout contains

$$300 \times 2048 = 614400$$

transitions.

Because PPO completes full rollouts, the actual number of training transitions can exceed the nominal requested training budget of five million timesteps.

5.5 Per-Environment Capacity Configuration

Each training environment retains its assigned ground-station rate during environment construction, reset, action-mask evaluation, and stepping.

This is necessary because the shared capacity functions read the current configuration when transmission is executed. Maintaining the override for the complete environment interaction ensures that the intended 1.5, 2.0, and 2.5 Mbps training regimes are actually represented during training.

5.6 Legacy and Unified PPO

The same DRL architecture can be trained using either the legacy or unified utility configuration.

The observation space, action space, action masking, transition process, policy architecture, and PPO hyperparameters remain unchanged.

In legacy mode,

$$\omega_Q = 1, \quad \omega_T = \omega_E = \omega_F = 0.$$

Legacy mode uses one linear coverage segment,

$$(1.0, 1.0),$$

does not normalize task weights, and uses reconstruction quality

$$s_q = 1 - \text{LPIPS}_q.$$

The legacy reconstruction-quality values for depths $q \in \{2, 4, 8, 16\}$ are

$$0.4430, \quad 0.4686, \quad 0.4829, \quad 0.4961.$$

Unified mode instead enables normalized task weights, downstream segmentation quality, concave coverage, tardiness, resource cost, and the final maximin fairness term.

This setup supports a controlled comparison between a PPO policy trained using the historical objective and a PPO policy trained using the unified objective.

6 Evaluation Protocol

6.1 Held-Out Task Seeds

Training uses task-generation seeds 0 through 99.

Evaluation uses held-out seeds

$$100, 101, \dots, 119.$$

These seeds generate task realizations that are not included in PPO training.

6.2 In-Distribution Evaluation

The nominal in-distribution evaluation uses

$$R_{\text{GS}} = 2.0 \text{ Mbps}.$$

This capacity lies within the set of rates presented during training.

6.3 Bandwidth Generalization

The policy can additionally be evaluated across

$$R_{\text{GS}} \in \{0.75, 1.0, 2.0, 3.0, 4.0\} \text{ Mbps.}$$

The 0.75, 1.0, 3.0, and 4.0 Mbps conditions are outside the exact set of training rates and can therefore be used to examine generalization to different bandwidth regimes.

6.4 Baseline Comparison

PPO policies are evaluated using the same satellite-network simulation and utility calculation used by the non-RL schedulers.

This permits matched comparisons with implemented scheduling strategies such as fixed-depth policies, greedy schedulers, and model-predictive-control methods.

Relevant evaluation quantities include:

- total unified utility,
- delivered fraction,
- quality contribution,
- tardiness contribution,
- resource-cost contribution,
- minimum normalized task utility,
- Jain’s fairness index.

7 Implementation Details and Current Limitations

7.1 Shared Network Execution

The DRL environment does not use a separate network model from the other schedulers.

After PPO commits a task to a compression depth, the task uses the shared route construction, edge-capacity accounting, propagation-delay calculation, and delivery-recording mechanisms.

This provides consistent execution semantics between the learned policy and the non-RL scheduling baselines.

A route consumes capacity from each traversed ISL, the final GSL, and the destination ground station’s aggregate receive budget.

7.2 Decision-Point Time Scale

An RL step does not necessarily correspond to one 30-second network slot.

The environment may execute already committed tasks and advance across network slots internally before reaching another task requiring an agent decision.

The agent therefore operates at task decision points layered on top of the fixed-slot network simulation.

7.3 Training Reward Versus Final Utility

The PPO training reward and final unified schedule utility are related but are not identical.

PPO receives the current task’s immediate realized utility change together with backlog-based potential shaping.

The final schedule-level evaluation additionally includes the global maximin fairness term

$$\omega_F |\mathcal{K}| \min_k \hat{U}_k.$$

Therefore, fairness affects the final unified score but is not directly provided as a per-step PPO reward.

7.4 Temporal Credit Assignment

After an agent action, the environment may automatically execute tasks whose depths were committed previously while advancing toward the next decision point.

The immediate reward term

$$\Delta J_t$$

is computed from the current task’s delivered utility around the explicit agent action. Utility produced by subsequent automatic transmissions is reflected in task state and final schedule evaluation, but it is not necessarily attributed directly to that action’s immediate ΔJ_t .

This is an implementation detail of the current temporal credit-assignment scheme.

7.5 Bandwidth Observability

The current observation does not explicitly contain the configured ground-station capacity

$$R_{GS},$$

nor does it directly expose residual capacities for individual links.

Consequently, two decision points with similar task and route features can produce similar observations even when they occur under substantially different bandwidth regimes.

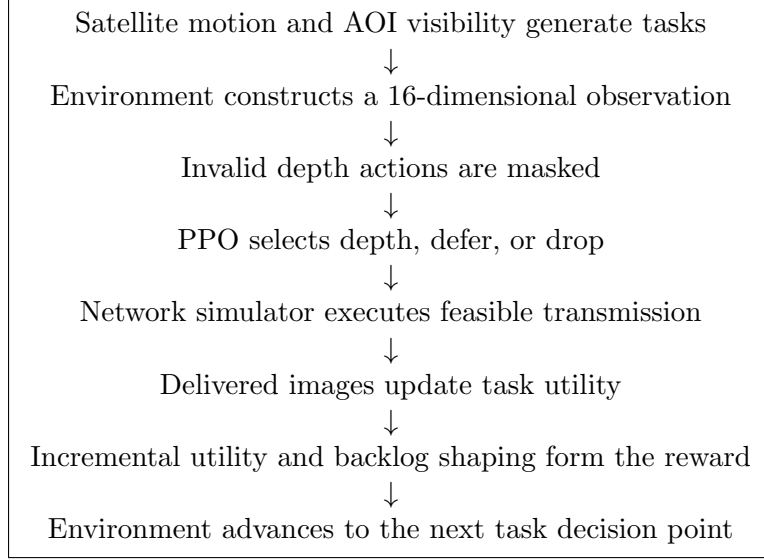
The policy receives indirect indicators of network conditions through route availability, route length, active-task count, destination contention, and freshness. These features do not, however, uniquely determine the available network capacity.

Explicit bandwidth, residual-capacity, or congestion features represent a possible extension for improving adaptation across substantially different network-capacity regimes.

8 Summary

The implemented DRL system separates learned task-level decision making from the physical satellite-network simulator.

The interaction can be summarized as



The learned policy determines compression-depth and admission decisions, while the satellite-network simulator determines orbital connectivity, routing, capacity-constrained transmission, encoding availability, propagation delay, and realized delivery outcomes.

The formulation therefore allows PPO to learn adaptive depth-selection behavior while preserving the same physical network execution model used by the existing scheduling baselines.